Trusted and Transparent Time-Stamping Through TEEs and Network Time Security

Tobias Nießen
TU Wien
Vienna, Austria
tobias.niessen@tuwien.ac.at

Andrew Paverd
MSRC
Cambridge, UK
andrew.paverd@microsoft.com

Abstract

Digital timestamps are essential for proving the existence of a digital document at a certain point in time. Time Stamp Authorities (TSAs) provide users with a mechanism for obtaining digital cryptographic signatures binding documents to points in time, which can be verified at a later time. However, this process requires that both the party obtaining the signature and the party verifying the signature trust the TSA's faithfulness, its time source, as well as its security practices. Conversely, operators of TSAs must themselves take extreme caution to meet these expectations. We devise a novel TSA-like architecture based on TEEs and show that this architecture, when coupled with access to a transparent set of secure time sources, can provide the user, the verifier, and the operator of the TSA with a high level of assurance.

CCS Concepts: • **Security and privacy → Trusted computing**; **Security services**; *Distributed systems security*.

Keywords: time stamp authorities, time-stamping, trusted computing, network time security, network time protocol

ACM Reference Format:
Tobias Nießen and Andrew Paverd. 2026. Trusted and Transparent Time-Stamping Through TEEs and Network Time Security. In *9th Workshop on System Software for Trusted Execution (SysTEX '26), April 27–30, 2026, Edinburgh, Scotland Uk.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3805690.3805731

1 Introduction

Modern systems increasingly rely on digital signatures to ensure data integrity, authenticity, and non-repudiation. Digital timestamps, as issued by Time Stamp Authorities (TSAs), are digital signatures that additionally cryptographically bind the signed document to the time at which the document was signed [19, 62]. In other words, the TSA vouches for the existence of the signed document at that time, which can be crucial in legal contexts. However, users of a TSA are entirely dependent on the availability of the TSA, its faithfulness, its time source, its security practices, as well as the long-term validity of its certificate chain. Similarly, TSA operators must take extreme precautions to ensure – beyond the usual operational security considerations of any PKI authority – the integrity, reliability, and accuracy of their time source [50].

Not entirely unlike randomness, real-world time cannot be computed; the passing of time can only be estimated through measurements, and once the current time has been estimated, it will already be in the past by the time the estimation is observed. In order to initialize and periodically synchronize the local system clock with a high-precision clock, such as a far away atomic clock, most systems use the Network Time Protocol (NTP) [37]. The local system then computes its local time estimate relative to the time obtained from one or more NTP servers while accounting for network delays. Unfortunately, NTP historically provided no security mechanism to ensure the authenticity or validity of the time being transmitted [37, 40] and even today, authentication is rarely used and man-in-the-middle attacks against NTP are both feasible and potentially impactful [23, 24, 36, 41, 44, 58]. While newer security mechanisms have the potential to prevent such attacks, assuming a significant increase in adoption, many vendors (including popular Linux distributions) rely on pools of volunteer-operated NTP servers, and a recent study demonstrated that a relatively small number of malicious NTP servers in a pool can manipulate NTP clients even in the presence of these new security mechanisms [48].

Despite the promising nature of Trusted Execution Environments (TEEs), it is widely accepted that TEEs lack a reliable sense of real-world time [3, 4, 20, 21, 59]. A TEE generally cannot trust the system clock, and any network traffic, such as NTP packets, may be intercepted by the host itself or by an external attacker. This is a known gap in confidential computing – the untrusted host might falsify time and the TEE would have no easy way to detect it. Even if a TEE had a trustworthy source of real-world time, it may be suspended or *rolled back* [26, 38] and thus perceive time incorrectly.

In this work, we demonstrate that Network Time Security (NTS), a modern security extension of NTP [18], can be combined with the security capabilities of TEEs to build high-assurance time-stamping mechanisms with direct traceability of timestamps to a transparent set of trusted, authoritative time sources. Bringing together TEEs and NTS for

This work is licensed under a Creative Commons Attribution 4.0 International License.
SysTEX '26, Edinburgh, Scotland Uk
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2607-1/26/04
https://doi.org/10.1145/3805690.3805731

time-stamping is a novel solution that pushes beyond existing methods and aligns with current trends of decentralizing trust and using hardware-based security. In doing so, we advance the state-of-the-art in two ways: (1) we extend the capabilities of TEEs to include trusted time-stamping; and (2) we leverage the emerging NTS protocol in a new context, demonstrating its value not just for synchronizing clocks but for building higher-level trust services.

2 Background

We briefly introduce relevant concepts and technologies.

2.1 Time Stamp Authorities (TSAs)

Time Stamp Authorities are services that provide trusted timestamps for digital documents, i.e., they produce cryptographic signatures that bind a digital document to the time at which the signature was issued [5, 19], and these signatures can later be verified against the TSA certificate. They are often used in legal, financial, and regulatory contexts where proving the time of the existence of a document is crucial, and have been incorporated into legal frameworks such as the EU's eIDAS regulation [15, 16]. Today, multiple standards cover protocols and operations of TSAs. Most prominently, RFC 3161 [62] defines the Time Stamp Protocol (TSP), while RFC 3628 [50] defines operational requirements for TSAs.

2.2 Trusted Execution Environments (TEEs)

Trusted Execution Environments are environments in which code can be executed in a secure and isolated manner by relying on hardware-based security features that guarantee the integrity and confidentiality of the code and data within them. While some TEE implementations are designed for fine-grained isolation and typically protect small, security-critical memory regions, other platforms allow running entire applications, containers, or virtual machines in isolation from the host system [1, 9, 22, 43, 45]. Importantly, TEE platforms provide *attestation* mechanisms that allow local or remote parties to verify the authenticity and integrity of the code and data within the enclave. While TEEs provide strong security guarantees about the code and data inside enclaves, the code inside a TEE generally cannot trust the system outside the enclave. Most I/O operations initiated by the enclave, such as reading from a file or network socket, are subject to external manipulation. Similarly, TEEs generally cannot trust the system clock. On some platforms, not only can the real-world time be manipulated by the host system (or simply be unintentionally inaccurate), but even measurements of elapsed time can be reset or otherwise tampered with [3, 4, 21, 59]. TEEs also cannot easily implement time keeping on their own, for one, because they generally do not have access to a trusted time source, and second, because they may be susceptible to rollback attacks, in which the host system reverts the TEE itself to a previous state [26, 38].

2.3 Network Time Protocol (NTP)

One of the oldest protocols on the internet is the Network Time Protocol (NTP), which is widely deployed across all major operating systems and used to synchronize clocks across devices [40]. End user devices typically act as NTP clients and obtain the current time from one or more NTP servers. Primary NTP servers are directly synchronized with a hardware time source, such as an atomic clock, and secondary NTP servers synchronize their clocks with one or more primary servers. The current version of NTP is NTPv4, which is specified in RFC 5905 [37].

2.4 Network Time Security (NTS)

NTP itself does not provide cryptographic guarantees w.r.t. the authenticity, integrity, or currentness of the time it provides, and thus allows adversaries to manipulate time synchronization in various ways [36]. However, some NTP servers support the use of symmetric cryptographic keys to authenticate the responses they provide, in which case the server includes a Message Authentication Code (MAC) in its responses. One such example is the *NIST Authenticated NTP Service*, which uses fixed symmetric keys that are associated with particular users [32, 33]. This allows users who have obtained a key to verify the authenticity and integrity of the time they receive from the server, but not its currentness.

In 2010, an asymmetric cryptographic protocol extension was added to NTP, which allows users to verify the authenticity of the responses they receive from an NTP server using public key cryptography [42]. However, this *Autokey* protocol has not been widely adopted due to several performance issues and security vulnerabilities [36], and current guidelines unconditionally advise against its use [52]. In 2016, another authenticated variant of NTP was proposed as a replacement for Autokey [13], but was not widely adopted.

Recently, in 2020, the Network Time Security (NTS) protocol has been standardized to provide improved security guarantees for NTP [18]. NTS protects the integrity and authenticity of NTP messages using modern cryptographic techniques, including the use of the most recent version of Transport Layer Security (TLS 1.3 [53]). NTS also provides a mechanism for users to detect replay attacks and to ensure that the NTP response they receive in response to a request is indeed current. Of course, each authentication mechanism incurs some overhead, both in terms of the computational cost of the cryptographic operations and in terms of increased message sizes. Nevertheless, multiple studies of the performance of NTP with NTS have shown that the overhead only has a small impact on precision in practice [28, 29, 31].

While these authentication mechanisms provide authenticity and integrity of the transmitted data, none of them provide non-repudiation guarantees. In other words, they do not allow an NTP client to prove that a particular time was provided by a particular NTP server or in response to

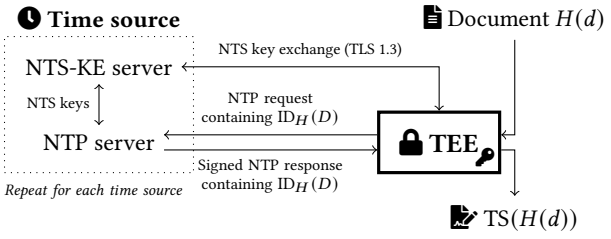

Figure 1. Architecture overview.

a particular NTP request [18, 32, 37, 42]. Asymmetric cryptographic algorithms are generally deemed unsuitable for authenticating individual NTP packets [41], which is why each mechanism exchanges symmetric cryptographic keys that are used for authenticating NTP packets containing time information, meaning that an NTP client that is able to verify the authenticity of an NTP packet using said symmetric key can also sign new NTP packets using the same symmetric key, as if they had been signed by the NTP server.

NTS relies on a secondary protocol for the exchange of these symmetric keys. This *NTS-KE* (NTS Key Exchange) protocol is built on top of TLS 1.3 [53] and utilizes standard TLS features for the certificate-based server authentication and the key exchange. The symmetric keys obtained as the result of a successful key exchange through NTS-KE are then used with NTS-specific extension fields that are added to each NTP request and response [18]. While TLS 1.3 does not provide non-repudiation [54], it does mandate *forward secrecy*, meaning that even a full compromise of the NTS-KE server's private key will not reveal previously negotiated symmetric keys [10, 53]. The lifetime of the symmetric keys is only limited by the upper bound on the number of cryptographic operations that may be performed using a single symmetric key, and even the expiration of the TLS certificate used during the NTS-KE protocol generally does not necessitate exchanging new keys [18, Section 8.5].

3 TEE-based Time-Stamping using NTS

We propose an alternative to traditional Time Stamp Authorities (TSAs) that leverages Trusted Execution Environments (TEEs) and Network Time Security (NTS) in order to establish cryptographically secure and trusted timestamps without relying on the faithfulness, accuracy, and integrity of a centralized TSA. The key idea is to use TEEs to securely obtain accurate and current time measurements from multiple NTP servers via NTS in such a way that these time measurements cannot be backdated, and to bind these claims to digital documents, enabling TEEs to create digital timestamps in a trustworthy and transparent manner. The private signing key used by the TEE for time-stamping is generated and kept within the TEE and attested to, such that the authenticity of timestamps can be verified directly against the TEE attestation. Figure 1 depicts key aspects of this architecture.

Threat model. *We assume that the host system running the TEE is untrusted. In particular, it may manipulate both the system time and any network traffic into and out of the TEE, and it may suspend the TEE for unknown periods of time. (Hardware-specific side-channel attacks against the TEE platform are out of scope for this work.) We further assume the existence of some* trusted *NTP time sources, which are expected to behave honestly, however, some or all of them may be impersonated through network manipulation. Despite time sources generally being trusted, we also want to maximize resilience against time sources malfunctioning or acting adversarially.*

As discussed in Section 2.4, NTS does not provide non-repudiation guarantees, and anyone in possession of the symmetric NTS-KE (TLS) session keys or of the derived NTS keys, which are required for verifying the authenticity of NTP messages, can also craft arbitrary messages with a valid signature. Our first observation is that TEEs allow us to effectively establish non-repudiation for NTS. The TEE's attestation to its own code and configuration means that we can trust the TEE to faithfully verify the certificate chain of the NTS-KE server. We rely on the hardware-backed memory confidentiality [22, 43] for keeping the private keys used during the NTS-KE handshake and the exchanged symmetric TLS and NTS keys private, which otherwise would allow an adversary to forge authenticated NTP responses. We can further verify that the code in the TEE is using these keys only as intended, to sign requests and to validate responses.

We will extend this mechanism such that the TEE provides evidence that for any issued timestamp, it has received corresponding NTP responses from trusted NTP servers and that it has successfully verified the authenticity of the messages, without ever revealing the keys that are used for said verification. These timestamps will then be signed using a private signing key, whose confidentiality is also backed by the underlying hardware. The TEE's attestation to its code and its timestamp signing key provides users with certainty that timestamps signed using this particular key were created in an honest and authentic manner.

3.1 Cryptographic Commitments against Backdating

For time-stamping, it is crucial that the signature was not backdated. In our case, there must be a strict happens-before relationship between the enclave seeing evidence of the existence of a document (in the form of a hash value $H(d)$ for a collision-resistant and preimage-resistant hash function H and a document d) and the NTP server issuing the NTP response that the timestamp will be based on. Our second crucial observation is that we can guarantee this happens-before relationship through cryptographic commitments as follows. While the enclave attests to its code and thus to its control

flow, which only requests the time from the time sources after it has received the document hash, this alone does not prevent an adversary from replaying an earlier NTP response with a valid NTS authenticator extension field. Fortunately, NTS provides a mechanism for preventing such replay attacks through the inclusion of a *unique identifier* in each NTP request that must be echoed in the authenticated NTP response. The NTS standard recommends that the unique identifier be a random bit sequence [18, Section 5.3]. Instead, we use a cryptographic commitment to the existence of the document as the unique identifier, which by construction is computationally indistinguishable from a random sequence. Given a collision-resistant and preimage-resistant hash function H and a document d, $H(d)$ is a cryptographic commitment to d. Further, given a set of documents D, we construct the cryptographic commitment $\mathrm{ID}_H(D)$ using a standard Merkle tree [39], whose leaves form the set $\{ H(d) \mid d \in D \}$. Using a Merkle tree allows grouping concurrent timestamp requests into a single set of NTP requests and thereby improve scalability while still ensuring the happens-before relationship across all timestamp requests and NTP responses.

This technique allows us to prove the happens-before relationship between receiving the document hash and the creation of the NTP response message: because the NTP response echoes the unique identifier, it contains the same cryptographic commitment $\mathrm{ID}_H(D)$ to the existence of the document(s) that was computed by the TEE. This cryptographic guarantee ensures that even complex and concurrent control flow, replay attacks, or network manipulation can never lead to a violation of the happens-before relationship and thus prevents the backdating of timestamps.

Note that the TEE never knows the contents of the document d, and it is indeed not necessary for the TEE to compute $H(d)$ itself. An honest user that computes $h = H(d)$ and sends it to the TEE obtains a timestamp for the document d, which must have existed in order for the user to be able to compute $H(d)$, and because H is collision-resistant, it will be infeasible to find any other document $d' \neq d$ with $H(d') = h$, i.e., a distinct document for which the same timestamp will be valid. A malicious user that sends a bit string h to the TEE that is *not* the result of applying H to some known (and thereby existing) document will obtain a timestamp but, because H is preimage-resistant, it will be infeasible to find *any* document d with $H(d) = h$.

3.2 NTS-KE Certificate Validation

The NTS-KE protocol requires the client to validate the server's TLS certificate chain. While this generally is a well-established procedure, it presents a challenge in our scenario, in which the TEE is expected to validate the certificate chain of the NTP server without relying on the host system's clock. This difficulty is acknowledged in the NTS specification [18, Section 8.5], which however only assumes that the system clock may be inaccurate or uninitialized but not that it may be manipulated by an active adversary. It is nonetheless true that the adversary only benefits from providing the client with an expired certificate if the same adversary has also compromised the private key associated with the certificate. Therefore, in either case, the specification's recommendation to use multiple independent NTS servers is an effective mitigation, as long as it remains unlikely for an adversary to successfully impersonate several trusted NTS-KE servers.

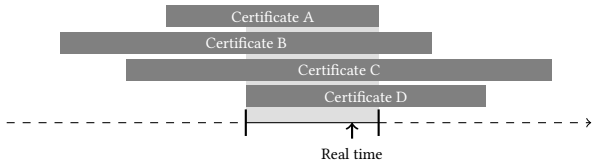

Figure 2. Certificate validity periods in normal conditions.

Leaf certificates used by TLS servers often have much shorter validity periods than intermediate and root certificates [34]. An honest NTS-KE server will use a TLS certificate that is valid at the time of the TLS handshake. Ideally, assuming that all NTS-KE handshakes are performed concurrently, the validity period of all NTS-KE certificates obtained during simultaneous TLS handshakes should overlap, and the true time at which the TLS handshake is performed should be within this intersection (see Figure 2). If a certificate's validity period does not overlap with that of certificates obtained from other NTS-KE servers during simultaneous handshakes, it is a strong indication of misconfiguration, malfunction, or malfeasance. Further, the NTP server is expected to respond with a time within the validity period of the NTS-KE server's TLS certificate if queried *immediately* after the NTS-KE handshake. By combining the information obtained during each of the NTS-KE handshakes and from the first NTP responses immediately after, the TEE can determine whether or not the time that it assumed during the TLS certificate validation is consistent with the certificate validity periods and NTP responses of all time sources.

Under the assumption that the TEE shall only issue timestamps if all time sources are working as expected, the TEE may simultaneously perform an NTS-KE handshake with each of the independent time sources. If the validity periods of the obtained TLS certificates do not overlap, this process is repeated. Subsequently, the TEE requests the current time immediately after each handshake from each NTP server while already relying on NTS for authentication. If any of the obtained times do not fall into the overlapping window of the certificate validity periods, the process is restarted. The host system's (untrusted) time may optionally be included in this process, and the TEE may refuse to issue timestamps if this time falls outside of the certificate overlap period or if it differs significantly from time measurements obtained from NTP servers via NTS.

3.3 NTP Eras

NTP communicates time as a fixed-point count of seconds since the beginning of the current *era*. Because NTP does not communicate the era, the protocol can represent arbitrary times within a fixed number of bits, but requires explicit knowledge of the current era in order to correctly determine the relevant offset [37]. In theory, the uncertainty of the current era presents a challenge for our design because the TEE cannot rely on the host's clock to determine the current era, and incorrectly determining the era could lead to backdated timestamps. In practice, however, the fact that every era spans roughly 136 years means that, for the foreseeable future, the TEE can safely assume that the current era is either era 0, which will end in 2036, or era 1, which will last until 2172. Upon receiving an NTP response, the TEE could simply assume era 0 if this offset would place the NTP time no earlier than the year 2026, and assume era 1 otherwise, and this assumption would be correct from the time of writing until the year 2162. It is not necessary to rely on such a crude approach: in Section 3.2, we assumed knowledge of the current era for the purpose of verifying that the time measurements obtained through NTP immediately after the (concurrent) NTS-KE handshakes fall within the overlapping validity periods of the NTS-KE certificates. Because this overlapping validity period will be much shorter than an NTP era and may only span a few weeks, only a single and unique NTP era will result in the success of this validation process, and this era must be the same across all NTP servers.

3.4 Toward TSA-like Time-Stamping

We have designed a mechanism that allows TEEs to provide evidence that a document existed *prior* to securely obtaining authentic, current, and non-repudiable time measurements from trusted NTP servers. This provides strong evidence of the existence of a document at a particular time. However, a traditional TSA implementing the Time Stamp Protocol (TSP) [62] must include a single time value in its timestamp response as opposed to a set of time measurements. To achieve such TSA-like behavior, there are various ways in which the time measurements can be combined to form a single timestamp. It is desirable, however, that this aggregation does not make the system susceptible to manipulation by a single *falseticking* time source, whether that is due to malfunction or malfeasance. For example, simply reporting the average of the time measurements would allow a single NTP server to significantly manipulate the reported time.

Due to the impossibility of communicating time with perfect accuracy, both NTP [37] and TSP [62] allow the respective server to specify the degree of *uncertainty* of the time included in their response. Each NTP response contains a time with an uncertainty estimate, which we will refer to as $t_i^{\mathrm{NTP}} \pm \Delta_i^{\mathrm{NTP}}$, where i is the index of the measurement among the NTP responses. We can aggregate these values into a single TSP-compatible time and accuracy $t^{\mathrm{TSP}} \pm \Delta^{\mathrm{TSP}}$ in several ways. First, we define a *maximally transparent* class of aggregation functions, whose members ensure that all time measurements obtained from the NTP servers are contained in the resulting interval $[t^{\mathrm{TSP}} - \Delta^{\mathrm{TSP}}, t^{\mathrm{TSP}} + \Delta^{\mathrm{TSP}}]$. Given a function that computes a single point in time t^{TSP} from a given set of n NTP time responses $t_i^{\mathrm{NTP}} \pm \Delta_i^{\mathrm{NTP}}$, let the uncertainty be $\Delta^{\mathrm{TSP}} = \max\{t_{\max} - t^{\mathrm{TSP}}, t^{\mathrm{TSP}} - t_{\min}\}$, where $t_{\min} = \min_i (t_i^{\mathrm{NTP}} - \Delta_i^{\mathrm{NTP}})$ and $t_{\max} = \max_i (t_i^{\mathrm{NTP}} + \Delta_i^{\mathrm{NTP}})$. We provide the following example instances:

- $\mathcal{T}_1$ sets $t^{\mathrm{TSP}} = \frac{1}{2}(t_{\min} + t_{\max})$, thereby minimizing Δ^{TSP}, but t^{TSP} may be significantly different from the actual time at which the NTP requests were made if just one of the responses is delayed or has low accuracy.
- $\mathcal{T}_2$ sets $t^{\mathrm{TSP}} = \frac{1}{n} \sum_{i \le n} t_i^{\mathrm{NTP}}$, i.e., to the arithmetic mean of t_i^{NTP}, but thereby increases Δ^{TSP} compared to $\mathcal{T}_1$.
- $\mathcal{T}_3$ sets $t^{\mathrm{TSP}} = \frac{1}{2}\left(t^{\mathrm{NTP}}_{\lceil n/2 \rceil} + t^{\mathrm{NTP}}_{\lceil (n+1)/2 \rceil}\right)$, where $t_i^{\mathrm{NTP}} \le t_{i+1}^{\mathrm{NTP}}$, which makes the point in time more robust against outliers at the cost of further increasing Δ^{TSP}.

We discuss the statistical behavior of these aggregation functions in the presence of network delays, uncertainty w.r.t. the true time, and adversarial behavior in Appendix A.

It is desirable for timestamps issued by a TSA to be monotonic in the sense that the order in which timestamps are issued matches the ordering of the associated time values t^{TSP}. Naturally, $\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$ exhibit a simple monotonicity property: if all t_i^{NTP} are non-decreasing and Δ_i^{NTP} are constant, then t^{TSP} is non-decreasing. $\mathcal{T}_2$ and $\mathcal{T}_3$ exhibit this monotonicity even if Δ_i^{NTP} are not constant. $\mathcal{T}_2$ satisfies a stronger monotonicity property: the value of t^{TSP} is non-decreasing as long as the *sum* of t_i^{NTP} is non-decreasing.

These *maximally transparent* methods for determining the timestamp time and uncertainty are faithful even if only one of the NTP servers reports the real time, since the real time will be within the interval $t^{\mathrm{TSP}} \pm \Delta^{\mathrm{TSP}}$. However, network delays affecting just a single NTP server can lead to a significantly degraded accuracy of the final timestamp. Robust NTP setups will typically use multiple NTP servers, with some guidelines recommending at least four different NTP servers [52, Section 3.2], which provides sufficient redundancy for detecting one *falseticker* even if one of the other three NTP servers becomes unavailable entirely. If there are strictly more than two NTP servers acting as time sources, it may be reasonable to assume that not just one of the servers will be faithful and accurate, but that fewer than half of the time sources are either unfaithful or inaccurate. Under that security assumption, a limited number of outliers can be removed from the set of time measurements before calculating the final timestamp and uncertainty (e.g., using Dixon's Q test, which works well for a small number of measurements [11, 55]), yet extra care must be taken to maintain monotonicity of the computed time in this case.

Based on our threat model, it is also crucial to balance resilience against a falseticking time source with resilience against an active adversary that may intercept and manipulate network traffic. In particular, the host system or an on-path attacker may interfere with the availability of NTP servers and may also delay NTP requests and responses. If such an adversary has managed to impersonate one of the NTP servers despite the use of NTS, the attacker may benefit from making other NTP servers unavailable to the TEE in order to make it more difficult for the TEE to determine that the impersonated NTP server is a falseticker, or it may benefit from delaying NTP requests to other servers such that their (delayed) responses will appear to be outliers. These issues can be mitigated through a carefully designed outlier detection mechanism as well as by refusing to issue a signed timestamp if too many NTP servers did not respond to NTP requests.

Because the exact choice of a particular policy for determining the timestamp time depends on various factors, our architecture does not assume a particular policy. The TEE attestation incorporates its configuration, including the set of NTS servers and the policy for calculating $t^{\mathrm{TSP}} \pm \Delta^{\mathrm{TSP}}$. This design allows for customization depending on the operator's threat model without sacrificing transparency or trust.

3.5 Choice of NTP Servers

Despite NTS being a recent standard and many NTP servers not yet providing the security guarantees that are necessary to support our approach, there already are sufficiently many public NTS-enabled NTP servers to allow for a robust time-stamping service. For example, the national metrology institute of Germany [49], who co-authored the NTS specification [18], the *Swedish Distributed Time Service* [14], and the Brazilian *Núcleo de Informação e Coordenação do Ponto BR* all operate government-affiliated NTS-enabled NTP servers that are directly synchronized with atomic clocks. Cloudflare, a company that handles a significant portion of the world's internet traffic, operates a globally distributed set of public NTS servers [35]. Further NTS servers are operated by universities (e.g., [51, 60]), companies, and individuals. Of course, operators of TEE-based time-stamping services may also choose to operate and include their own NTP servers alongside publicly trusted and auditable time servers.

The operator, location, and physical reference clock of the server may be the most important but not the only criteria to consider when choosing NTP servers for time-stamping. Because NTS-KE relies on TLS, its security depends on the TLS certificate chains and associated keys not being compromised. While the compromise of a certificate's private key does not invalidate or compromise the security of past NTS-KE handshakes, the exact time of compromise may be unknown and/or certificate revocations may not be published immediately. Ideally, the certificate chains of the different NTS servers should have distinct sets of intermediate and even root certificates, so that a compromise of any single certificate does not affect the security of all NTS servers.

4 Integration with Existing Infrastructure

Our work allows the creation of Time Stamp Authorities with a higher degree of trust, transparency, and decentralization than traditional TSAs. Our time-stamping approach permits full transparency into its time sources, and those NTP time sources that are publicly available can be independently monitored for their accuracy. Because our design does not visibly deviate from the normal use of NTP/NTS, it would be difficult for time servers to identify that they are being used for time-stamping, and therefore, they cannot easily manipulate the time they provide to our TEE without risking that the same manipulation may be detected by other clients.

Similarly, the operator of a TSA based on our design can achieve a high degree of confidence in their own service and can incorporate their own time sources alongside publicly auditable time sources if desired, with the TEE attesting to a policy that provides both the operator of the TSA and its users with a high degree of trust in the resulting timestamps.

To further increase the trust in a TEE-based TSA, it is important to ensure that the TSA operator cannot issue additional timestamps that were not created from within a compliant TEE. The timestamp signing keys are generated and attested to by the TEE itself, so the TSA operator cannot use these keys to issue timestamps except through the TEE's regular operation. For a signing key to be trusted, the corresponding (leaf) certificate must be signed using the private key associated with an intermediate or root certificate, and the respective Certificate Authority (CA) can verify the attestation before issuing the leaf certificate for the TEE (e.g., as per [7, 47]). The Certificate Authority should further record all issued certificates in a public Certificate Transparency (CT) log [30, 56]. A TSA operator following this procedure must then provide TEE attestation evidence for each issued certificate, and any signature that cannot be verified against a certificate from the CT log or for which the operator cannot produce attestation evidence would reveal that the TSA used a signing key not belonging to a compliant TEE.

5 Related Work

Town Crier [61], similar to our own work, uses TEEs to confidentially perform TLS handshakes – albeit for the HTTPS protocol and not for the NTS-KE protocol – and thereby ensures the authenticity and, in some sense, non-repudiation of the server's HTTP response, but assumes that the TEE has access to a trustworthy clock for certificate validation.

Multiple recent works have discussed the difficulties of establishing a trusted notion of time in TEEs. S-FaaS [2] establishes lower bounds on the amount of elapsed time within a TEE, but no upper bounds that would be required for time-stamping. Déjà vu [8] implements a monotonic

reference clock for Intel SGX with protection against adversarial manipulation but this counter cannot be directly translated into real-world clock time. T3E [20], TimeSeal [3], TimeGuard [46], Triad [17], and TriHaRd [6] also provide mechanisms for measuring elapsed time within a TEE with bounded error and resilience against adversarial manipulation, with varying security guarantees and threat models. However, these approaches all depend on intricate details of platform-specific security features, whereas our design is compatible with a wide range of TEE architectures since it requires only the confidentiality and integrity of the enclave's memory as well as a secure attestation mechanism.

On the NTP side, while NTS [18] provides origin authentication, Khronos [12, 25, 57] is a recently published mechanism for hardening NTP against *malicious* falsetickers.

Roughtime [27] may in the future aid certificate verification by securely providing approximate time information to clients which could then be compared to NTS-KE certificate validity periods. At the time of writing, the development of Roughtime is ongoing and its adoption is hindered by a lack of infrastructure and key management mechanisms.

6 Conclusion and Future Work

In this work, we have presented a novel TSA architecture that combines Trusted Execution Environments with the recently standardized Network Time Security [18] extension of the Network Time Protocol [37] in order to issue digitally signed timestamps in a transparent, trustworthy, and potentially decentralized manner, with timestamps being directly traceable to a set of trusted time sources, while maintaining interoperability with the existing Time Stamp Protocol [62], which is mandatory in legal frameworks such as the EU's eIDAS regulation [15, 16]. This architecture can be transparently adjusted based on the operator's threat model and security assumptions by including an operator-defined policy in the TEE attestation.

Our design requires one or more NTP queries in-between receiving the document hash and producing the signed timestamp. Potential scalability issues for TSAs that need to quickly issue a large number of timestamps are addressed by grouping concurrent requests (see Section 3.1). A promising future direction is to combine our architecture with emerging techniques for securely measuring elapsed time from within TEEs [3, 6, 17, 20] to reduce the frequency of NTP queries without sacrificing the traceability of timestamps to trusted time sources and hardware-backed security.

Acknowledgments

The project leading to this application has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 101034440.

References

[1] Agence nationale de la sécurité des systèmes d'information (ANSSI). 2025. *Technical Position Paper on Confidential Computing*. Position paper. ANSSI.

[2] Fritz Alder, N. Asokan, Arseny Kurnikov, Andrew Paverd, and Michael Steiner. 2019. S-FaaS: Trustworthy and Accountable Function-as-a-Service using Intel SGX. In *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop* (London, United Kingdom) *(CCSW'19)*. Association for Computing Machinery, New York, NY, USA, 185–199. doi:10.1145/3338466.3358916

[3] Fatima M. Anwar, Luis Garcia, Xi Han, and Mani Srivastava. 2019. Securing Time in Untrusted Operating Systems with TimeSeal. In *2019 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, Hong Kong, China, 80–92. doi:10.1109/RTSS46320.2019.00018

[4] Fatima M. Anwar and Mani Srivastava. 2019. Applications and Challenges in Securing Time. In *12th USENIX Workshop on Cyber Security Experimentation and Test (CSET 19)*. USENIX Association, Santa Clara, CA, 5 pages. https://www.usenix.org/conference/cset19/presentation/anwar

[5] Dave Bayer, Stuart Haber, and W. Scott Stornetta. 1993. Improving the Efficiency and Reliability of Digital Time-Stamping. In *Sequences II*, Renato Capocelli, Alfredo De Santis, and Ugo Vaccaro (Eds.). Springer New York, New York, NY, 329–334. doi:10.1007/978-1-4613-9323-8_24

[6] Matthieu Bettinger, Sonia Ben Mokhtar, Pascal Felber, Etienne Rivière, Valerio Schiavoni, and Anthony Simonet-Boulogne. 2025. TriHaRd: Higher Resilience for TEE Trusted Time. arXiv:2512.10732 [cs.CR] https://arxiv.org/abs/2512.10732

[7] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334. RFC Editor. doi:10.17487/RFC9334

[8] Sanchuan Chen, Xiaokuan Zhang, Michael K. Reiter, and Yinqian Zhang. 2017. Detecting Privileged Side-Channel Attacks in Shielded Execution with Déjà Vu. In *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security* (Abu Dhabi, United Arab Emirates) *(ASIA CCS '17)*. Association for Computing Machinery, New York, NY, USA, 7–18. doi:10.1145/3052973.3053007

[9] Pau-Chen Cheng, Wojciech Ozga, Enriquillo Valdez, Salman Ahmed, Zhongshu Gu, Hani Jamjoom, Hubertus Franke, and James Bottomley. 2024. Intel TDX Demystified: A Top-Down Approach. *ACM Comput. Surv.* 56, 9, Article 238 (April 2024), 33 pages. doi:10.1145/3652597

[10] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. 2017. A Comprehensive Symbolic Analysis of TLS 1.3. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS '17)*. Association for Computing Machinery, Dallas, Texas, USA, 1773–1788. doi:10.1145/3133956.3134063

[11] R. B. Dean and W. J. Dixon. 1951. Simplified Statistics for Small Numbers of Observations. *Analytical Chemistry* 23, 4 (1951), 636–638. doi:10.1021/ac60052a025

[12] Omer Deutsch, Neta Rozen Schiff, Danny Dolev, and Michael Schapira. 2018. Preventing (Network) Time Travel with Chronos. In *25th Annual Network and Distributed System Security Symposium, NDSS*. The Internet Society, San Diego, California, USA, 15 pages.

[13] Benjamin Dowling, Douglas Stebila, and Greg Zaverucha. 2016. Authenticated Network Time Synchronization. In *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, Austin, TX, 823–840. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/dowling

[14] Sven-Christian Ebenhag, Per Olof Hedekvist, Per Jarlemark, and Ragnar Sundblad. 2019. Redundant Distributed Timescale Traceable to UTC(SP). In *2019 Joint Conference of the IEEE International Frequency Control Symposium and European Frequency and Time Forum (EFTF/IFC)*. IEEE, Orlando, FL, USA, 1–4. doi:10.1109/FCS.2019.8856058

[15] European Commission. 2025. Commission Implementing Regulation (EU) 2025/1929 of 29 September 2025 laying down rules for the application of Regulation (EU) No 910/2014 of the European Parliament and of the Council as regards the binding of date and time to data and establishing the accuracy of the time sources for the provision of qualified electronic time stamps. Official Journal of the European Union. http://data.europa.eu/eli/reg_impl/2025/1929/oj

[16] European Parliament and Council of the European Union. 2014. Regulation (EU) No 910/2014 of 23 July 2014 on electronic identification and trust services for electronic transactions in the internal market. Official Journal of the European Union, 73-114 pages. http://data.europa.eu/eli/reg/2014/910/oj

[17] Gabriel Fernandez, Andrey Brito, and Christof Fetzer. 2023. Triad: Trusted Timestamps in Untrusted Environments. In *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE, Naples, Italy, 169–176. doi:10.1109/CloudCom59040.2023.00037

[18] Daniel Fox Franke, Dieter Sibold, Kristof Teichel, Marcus Dansarie, and Ragnar Sundblad. 2020. *Network Time Security for the Network Time Protocol*. RFC 8915. RFC Editor. doi:10.17487/RFC8915

[19] Stuart Haber and W. Scott Stornetta. 1990. How to Time-Stamp a Digital Document. In *Advances in Cryptology - CRYPTO '90, 10th Annual International Cryptology Conference (Lecture Notes in Computer Science, Vol. 537)*, Alfred Menezes and Scott A. Vanstone (Eds.). Springer, Santa Barbara, California, USA, 437–455. doi:10.1007/3-540-38424-3_32

[20] Gilang Mentari Hamidy, Pieter Philippaerts, and Wouter Joosen. 2023. T3E: A Practical Solution to Trusted Time in Secure Enclaves. In *Network and System Security*, Shujun Li, Mark Manulis, and Atsuko Miyaji (Eds.). Springer Nature Switzerland, Cham, 305–326. doi:10.1007/978-3-031-39828-5_17

[21] Wei Huang, Shengjie Xu, Yueqiang Cheng, and David Lie. 2021. Aion Attacks: Manipulating Software Timers in Trusted Execution Environment. In *Detection of Intrusions and Malware, and Vulnerability Assessment*, Leyla Bilge, Lorenzo Cavallaro, Giancarlo Pellegrino, and Nuno Neves (Eds.). Springer International Publishing, Cham, 173–193. doi:10.1007/978-3-030-80825-9_9

[22] Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stapf. 2020. Trusted Execution Environments: Properties, Applications, and Challenges. *IEEE Security & Privacy* 18, 2 (2020), 56–60. doi:10.1109/MSEC.2019.2947124

[23] Philipp Jeitner, Haya Shulman, and Michael Waidner. 2020. The Impact of DNS Insecurity on Time. In *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. IEEE, Valencia, Spain, 266–277. doi:10.1109/DSN48063.2020.00043

[24] Philipp Jeitner, Haya Shulman, and Michael Waidner. 2020. Pitfalls of Provably Secure Systems in Internet - The Case of Chronos-NTP. In *2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. IEEE, Valencia, Spain, 49–50. doi:10.1109/DSN-S50200.2020.00027

[25] Philipp Jeitner, Haya Shulman, and Michael Waidner. 2020. Secure Consensus Generation with Distributed DoH. In *2020 50th Annual IEEE-IFIP International Conference on Dependable Systems and Networks-Supplemental Volume (DSN-S)*. IEEE, Valencia, Spain, 41–42. doi:10.1109/DSN-S50200.2020.00023

[26] Sadegh Keshavarzi, Gregory Chockler, and Alexey Gotsman. 2025. TEE Is Not a Healer: Rollback-Resistant Reliable Storage. In *39th International Symposium on Distributed Computing (DISC 2025) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 356)*, Dariusz R. Kowalski (Ed.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 39:1–39:18. doi:10.4230/LIPIcs.DISC.2025.39

[27] Watson Ladd and Marcus Dansarie. 2026. *Roughtime*. Internet-Draft draft-ietf-ntp-roughtime-19. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-ntp-roughtime/19/ Work in Progress.

[28] Martin Langer, Rainer Bermbach, Kristof Teichel, and Dieter Sibold. 2019. Performance Comparison Between Network Time Security Protocol Drafts: Improvements and Accuracy of the Latest NTS Draft. In *2019 Joint Conference of the IEEE International Frequency Control Symposium and European Frequency and Time Forum (EFTF/IFC)*. IEEE, Orlando, FL, USA, 1–7. doi:10.1109/FCS.2019.8856019

[29] Martin Langer, Kristof Teichel, Dieter Sibold, and Rainer Bermbach. 2018. Time synchronization performance using the network time security protocol. In *2018 European Frequency and Time Forum (EFTF)*. IEEE, Turin, 138–144. doi:10.1109/EFTF.2018.8409017

[30] Ben Laurie, Eran Messeri, and Rob Stradling. 2021. *Certificate Transparency Version 2.0*. RFC 9162. RFC Editor. doi:10.17487/RFC9162

[31] Griffin Leclerc and Radim Bartos. 2022. Experimental Analysis of the Performance and Scalability of Network Time Security for the Network Time Protocol. In *2022 IEEE International Symposium on Precision Clock Synchronization for Measurement, Control, and Communication (ISPCS)*. IEEE, Vienna, Austria, 1–7. doi:10.1109/ISPCS55791.2022.9918378

[32] Judah Levine. 1999. Authenticating time and frequency signals. In *Proceedings of the 1999 Joint Meeting of the European Frequency and Time Forum and the IEEE International Frequency Control Symposium (Cat. No.99CH36313)*, Vol. 1. IEEE, Besancon, France, 304–308. doi:10.1109/FREQ.1999.840767

[33] Judah Levine. 2015. NIST Internet Time Service. https://data.nist.gov/od/id/25C39B1D2C1549AFE0531A5706812CBC1445. Accessed: 2026-01-20.

[34] Zane Ma, Aaron Faulkenberry, Thomas Papastergiou, Zakir Durumeric, Michael D. Bailey, Angelos D. Keromytis, Fabian Monrose, and Manos Antonakakis. 2023. Stale TLS Certificates: Investigating Precarious Third-Party Access to Valid TLS Keys. In *Proceedings of the 2023 ACM on Internet Measurement Conference* (Montreal QC, Canada) *(IMC '23)*. Association for Computing Machinery, New York, NY, USA, 222–235. doi:10.1145/3618257.3624802

[35] Aanchal Malhotra. 2019. Introducing time.cloudflare.com. https://blog.cloudflare.com/secure-time/. Accessed 2026-01-19.

[36] Aanchal Malhotra, Isaac E. Cohen, Erik Brakke, and Sharon Goldberg. 2016. Attacking the Network Time Protocol. In *23rd Annual Network and Distributed System Security Symposium, NDSS*. The Internet Society, San Diego, California, USA, 15 pages. doi:10.14722/ndss.2016.23090

[37] Jim Martin, Jack Burbank, William Kasch, and David L. Mills. 2010. *Network Time Protocol Version 4: Protocol and Algorithms Specification*. RFC 5905. RFC Editor. doi:10.17487/RFC5905

[38] Sinisa Matetic, Mansoor Ahmed, Kari Kostiainen, Aritra Dhar, David Sommer, Arthur Gervais, Ari Juels, and Srdjan Capkun. 2017. ROTE: Rollback Protection for Trusted Execution. In *26th USENIX Security Symposium (USENIX Security 17)*. USENIX Association, Vancouver, BC, 1289–1306. https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/matetic

[39] Ralph C. Merkle. 1990. A Certified Digital Signature. In *Advances in Cryptology — CRYPTO' 89 Proceedings*, Gilles Brassard (Ed.). Springer New York, New York, NY, 218–238. doi:10.1007/0-387-34805-0_21

[40] David L. Mills. 1985. *Network Time Protocol (NTP)*. RFC 958. RFC Editor. doi:10.17487/RFC0958

[41] David L. Mills. 2011. *Computer Network Time Synchronization: the Network Time Protocol on Earth and in space* (2nd ed.). CRC Press.

[42] David L. Mills and Brian Haberman. 2010. *Network Time Protocol Version 4: Autokey Specification*. RFC 5906. RFC Editor. doi:10.17487/RFC5906

[43] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2024. Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX. *Proc. ACM Meas. Anal. Comput. Syst.* 8, 3, Article 36 (Dec. 2024), 42 pages. doi:10.1145/3700418

[44] Tal Mizrahi. 2014. *Security Requirements of Time Protocols in Packet Switched Networks*. RFC 7384. RFC Editor. doi:10.17487/RFC7384

[45] Antonio Muñoz, Ruben Ríos, Rodrigo Román, and Javier López. 2023. A survey on the (in)security of trusted execution environments. *Computers & Security* 129 (2023), 103180. doi:10.1016/j.cose.2023.103180

[46] Adeel Nasrullah and Fatima M Anwar. 2024. Trusted Timing Services with TimeGuard. In *2024 IEEE 30th Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, Hong Kong, Hong Kong, 1–14. doi:10.1109/RTAS61025.2024.00009

[47] Mike Ounsworth, Hannes Tschofenig, Henk Birkholz, Monty Wiseman, and Ned Smith. 2026. *Use of Remote Attestation with Certification Signing Requests*. Internet-Draft draft-ietf-lamps-csr-attestation-23. Internet Engineering Task Force. https://datatracker.ietf.org/doc/draft-ietf-lamps-csr-attestation/23/ Work in Progress.

[48] Yarin Perry, Neta Rozen Schiff, and Michael Schapira. 2021. A Devil of a Time: How Vulnerable is NTP to Malicious Timeservers?. In *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society, Virtual, 18 pages. doi:10.14722/ndss.2021.24302

[49] Physikalisch-Technische Bundesanstalt. [n. d.]. Time synchronization of computers using the "Network Time Protocol" (NTP). https://www.ptb.de/cms/en/ptb/fachabteilungen/abt9/gruppe-95/ref-952/time-synchronization-of-computers-using-the-network-time-protocol-ntp.html. Accessed 2026-01-19.

[50] Nick Pope, John Ross, and Denis Pinkas. 2003. *Policy Requirements for Time-Stamping Authorities (TSAs)*. RFC 3628. RFC Editor. doi:10.17487/RFC3628

[51] Regionales Rechenzentrum Erlangen (RRZE). [n. d.]. Zeitserver. https://www.rrze.fau.de/serverdienste/infrastruktur/zeitserver/. Accessed 2026-01-19.

[52] Denis Reilly, Harlan Stenn, and Dieter Sibold. 2019. *Network Time Protocol Best Current Practices*. RFC 8633. RFC Editor. doi:10.17487/RFC8633

[53] Eric Rescorla. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446. RFC Editor. doi:10.17487/RFC8446

[54] Hubert Ritzdorf, Karl Wust, Arthur Gervais, Guillaume Felley, and Srdjan Capkun. 2018. TLS-N: Non-repudiation over TLS enabling ubiquitous content signing. In *Proceedings of the Network and Distributed System Security Symposium 2018*. The Internet Society, San Diego, CA, 15 pages. doi:10.14722/ndss.2018.23272

[55] David B. Rorabacher. 1991. Statistical treatment for rejection of deviant values: critical values of Dixon's "Q" parameter and related subrange ratios at the 95% confidence level. *Analytical Chemistry* 63, 2 (1991), 139–146. doi:10.1021/ac00002a010

[56] Quirin Scheitle, Oliver Gasser, Theodor Nolte, Johanna Amann, Lexi Brent, Georg Carle, Ralph Holz, Thomas C. Schmidt, and Matthias Wählisch. 2018. The Rise of Certificate Transparency and Its Implications on the Internet Ecosystem. In *Proceedings of the Internet Measurement Conference 2018 (IMC '18)*. Association for Computing Machinery, Boston, MA, USA, 343–349. doi:10.1145/3278532.3278562

[57] Neta Rozen Schiff, Danny Dolev, Tal Mizrahi, and Michael Schapira. 2024. *A Secure Selection and Filtering Mechanism for the Network Time Protocol with Khronos*. RFC 9523. RFC Editor. doi:10.17487/RFC9523

[58] Jose Selvi. 2014. Bypassing HTTP Strict Transport Security. Black Hat Europe, 4 pages.

[59] Bohdan Trach, Rasha Faqeh, Oleksii Oleksenko, Wojciech Ozga, Pramod Bhatotia, and Christof Fetzer. 2020. T-Lease: a trusted lease primitive for distributed systems. In *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC '20)*. Association for Computing Machinery, Virtual Event, USA, 387–400. doi:10.1145/3419111.3421273

[60] University of Cambridge. 2026. Details of the Network Time Protocol service – University Information Services. https://help.uis.cam.ac.uk/service/network-services/network-time-protocol-ntp/details-of-the-network-time-protocol-service. Accessed 2026-03-23.

[61] Fan Zhang, Ethan Cecchetti, Kyle Croman, Ari Juels, and Elaine Shi. 2016. Town Crier: An Authenticated Data Feed for Smart Contracts. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security* (Vienna, Austria) *(CCS '16)*. Association for Computing Machinery, New York, NY, USA, 270–282. doi:10.1145/2976749.2978326

[62] Robert Zuccherato, Patrick Cain, Carlisle Adams, and Denis Pinkas. 2001. *Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP)*. RFC 3161. RFC Editor. doi:10.17487/RFC3161

Open Science

This work proposes a design of a novel trusted and transparent time-stamping architecture based on Trusted Execution Environments. No experimental results have been presented. The statistical analysis in Appendix A can be reproduced as per the below description.

A Statistical Comparison of Time Measurement Aggregation Functions

In Section 3.4, we discussed aggregation functions for determining the time and uncertainty of the timestamp being issued based on a set of time and accuracy claims from multiple NTP time sources. Recall that our goal is to compute some time and uncertainty $t^{\text{TSP}} \pm \Delta^{\text{TSP}}$ from a set of concurrent time measurements obtained from multiple NTP servers. For the class of *maximally transparent* aggregation functions, we compute from a given set of n NTP time responses $t_i^{\text{NTP}} \pm \Delta_i^{\text{NTP}}$ and some point in time t^{TSP} the uncertainty as

$$\begin{aligned}\Delta^{\text{TSP}} &= \max\{t_{\max} - t^{\text{TSP}}, t^{\text{TSP}} - t_{\min}\}, \text{ where}\\ t_{\min} &= \min_{1\le i\le n}\,(t_i^{\text{NTP}} - \Delta_i^{\text{NTP}}) \text{ and}\\ t_{\max} &= \max_{1\le i\le n}\,(t_i^{\text{NTP}} + \Delta_i^{\text{NTP}}).\end{aligned}$$

We presented three example instantiations for determining the value of t^{TSP}, which we recite here.

- $\mathcal{T}_1$ sets $t^{\text{TSP}} = \frac{1}{2}(t_{\min} + t_{\max})$, thereby minimizing the uncertainty measure Δ^{TSP}, but t^{TSP} may be significantly different from the actual time at which the NTP requests were made if just one of the responses is delayed or has very low accuracy.
- $\mathcal{T}_2$ sets $t^{\text{TSP}} = \frac{1}{n}\sum_{i\le n} t_i^{\text{NTP}}$, i.e., to the arithmetic mean of t_i^{NTP}, but thereby increases Δ^{TSP} compared to $\mathcal{T}_1$.
- $\mathcal{T}_3$ sets $t^{\text{TSP}} = \frac{1}{2}\left(t^{\text{NTP}}_{\lceil n/2\rceil} + t^{\text{NTP}}_{\lceil (n+1)/2\rceil}\right)$, where $t_i^{\text{NTP}} \le t_{i+1}^{\text{NTP}}$, which is the median of the time measurements and hence makes the point in time more robust against outliers at the cost of further increasing Δ^{TSP}.

Here, we provide a brief statistical comparison of these aggregation methods based on a realistic simulation of network delays in the communication with NTP servers and uncertainty in time estimates provided by these servers.

Beyond the class of *maximally transparent* aggregation functions, Section 3.4 also suggested the use of outlier detection to improve the accuracy of the resulting timestamp. In line with this suggestion, for the purpose of this evaluation, we also consider variants $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$ that each apply Dixon's Q test [11] to remove at most one outlier from the set of NTP time measurements before applying $\mathcal{T}_1$, $\mathcal{T}_2$, or $\mathcal{T}_3$ to the remaining measurements, respectively. We use a standard confidence level of $\alpha = 95\%$ for Dixon's Q test for a single outlier, commonly known as Q_{10}, resulting in critical values $\text{Crit}(\alpha, n)$ greater than 0.5 for $\alpha = 0.95$ and $n \le 8$ [55], where n is the number of NTP responses, which means that the outlier can be uniquely identified (if one exists). In other words, assuming that the NTP responses are ordered such that $t_i^{\text{NTP}} \le t_{i+1}^{\text{NTP}}$, at most one of these is true:

$$\frac{t_2^{\text{NTP}} - t_1^{\text{NTP}}}{t_n^{\text{NTP}} - t_1^{\text{NTP}}} > \text{Crit}(\alpha, n), \tag{1}$$

$$\frac{t_n^{\text{NTP}} - t_{n-1}^{\text{NTP}}}{t_n^{\text{NTP}} - t_1^{\text{NTP}}} > \text{Crit}(\alpha, n) \tag{2}$$

If Equation 1 or Equation 2 is true, the NTP response with the lowest or greatest value of t_i^{NTP} is discarded, respectively. Otherwise, the original (maximally transparent) aggregation function is applied to the full set of NTP responses. This use of outlier detection is faithful, in the sense that the interval $t^{\text{TSP}} \pm \Delta^{\text{TSP}}$ contains the true time, as long as at least two of the $n \ge 3$ NTP servers respond with (an approximation of) the true time. For setups that use more than three NTP servers, it may even be desirable to remove more than one outlier, but that is out of scope for this simulation.

A.1 Methodology

For each time measurement obtained from a (simulated) NTP server, we apply random noise to the measurement by shifting the time value by an amount drawn at random from a normal distribution (with mean $\mu = 0$ and parametric variance). Further, we emulate a network delay for each simulated NTP request, which we draw from from either a log-normal distribution or from a Pareto (Type I) distribution. The network delay distributions for the simulated NTP requests are listed in Table 1. The network delay of the NTP *response* is irrelevant for this comparison because it does not affect the computed time and accuracy.

We now briefly describe the precise simulation conditions such that the results may be independently reproduced. Let $\textsc{Uniform}(I)$ be a random variable uniformly distributed over the interval $I \subset \mathbb{R}$. Given a probabilistic network delay distribution, a maximum uncertainty $\Delta^{\max}$ (in seconds), and the true time $t^{\text{true}} \in \mathbb{R}$ as the number of seconds relative to some fixed offset (e.g., the beginning of the current NTP era), let $\textsc{Delay}$ be a random variable distributed according to the chosen network delay distribution (see Table 1), and let $\textsc{Noise}(\Delta)$ be a normal random variable with mean $\mu = 0$ and standard deviation $\sigma = \Delta/\Phi^{-1}(0.975)$, so that $\Pr(|\textsc{Noise}(\Delta)| \le \Delta) = 0.95$. For the purpose of this simulation, we then sample times and accuracies for the simulated NTP server responses as $t^{\text{NTP}} \pm \Delta^{\text{NTP}}$, where $\Delta^{\text{NTP}} = \textsc{Uniform}([0, \Delta^{\max}])$ and

$$t^{\text{NTP}} = t^{\text{true}} + \textsc{Noise}\left(\Delta^{\text{NTP}}\right) + \textsc{Delay}.$$

In each iteration, we obtain n such samples for $t^{\text{NTP}} \pm \Delta^{\text{NTP}}$ to simulate n NTP responses and we apply each aggregation function ($\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$ as well as their variants $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$) to these samples to obtain a time and uncertainty $t^{\text{TSP}} \pm \Delta^{\text{TSP}}$. We then report the absolute error $|t^{\text{TSP}} - t^{\text{true}}|$ and the uncertainty Δ^{TSP} (lower is better for both metrics). We simulate each combination for one million iterations.

The parts of the evaluation described thus far concern benign behavior only, i.e., there may be network delays and imprecision/uncertainty, but no adversarial behavior affects the calculation of t^{NTP} in the formalism defined above. To analyze resilience against adversarial behavior, we further simulate three additional scenarios, which are emulated in

Ref.	Distribution	25th perc.	50th perc.	75th perc.	95th perc.	99th perc.
A	Log-normal distribution ($\mu = \ln 0.025, \sigma = 2$)	0.006 s	0.025 s	0.096 s	0.671 s	2.622 s
B	Log-normal distribution ($\mu = \ln 0.250, \sigma = 1.5$)	0.091 s	0.250 s	0.688 s	2.948 s	8.193 s
C	Log-normal distribution ($\mu = \ln 0.500, \sigma = 1$)	0.255 s	0.500 s	0.982 s	2.590 s	5.120 s
D	Pareto distribution (min = 0.010, $\alpha = 0.75$)	0.015 s	0.025 s	0.063 s	0.543 s	4.642 s
E	Pareto distribution (min = 0.100, $\alpha = 1$)	0.133 s	0.200 s	0.400 s	2.000 s	10.000 s
F	Pareto distribution (min = 0.200, $\alpha = 1.5$)	0.242 s	0.317 s	0.504 s	1.474 s	4.309 s

Table 1. Network delay percentiles for the distributions used in the simulation. The value for the nth-percentile is the delay, in seconds, such that n percent of NTP requests are delayed by at most this amount of time.

the same manner as described above with the following distinct modifications:

- $\mathcal{A}_1$ An adversary is successfully impersonating one of the NTP servers and attempting to backdate timestamps by responding with a time that is ten seconds earlier than the time that the NTP server would have responded with otherwise.
- $\mathcal{A}_2$ In this extension of scenario $\mathcal{A}_1$, the attacker additionally delays the NTP request to one of the other NTP servers by three seconds.
- $\mathcal{A}_3$ In this extension of scenario $\mathcal{A}_1$, the attacker additionally delays the NTP request to one of the other NTP servers by fifty seconds.

While the durations used in these adversarial experiments may appear arbitrary or selective, we will see below that these choices demonstrate different consequences of adversarial behavior across different aggregation functions and across different values of n well. We do not explicitly model adversarial scenarios in which an attacker makes one or more NTP servers unavailable to the TEE, either as the malicious host of the TEE or as an on-path attacker. In this case, the TEE's time-stamping policy either prevents the issuance of a timestamp or it delegates to a sub-policy for the smaller value of n, in which case the results of this evaluation for the new value of n apply.

A.2 Analysis

We first analyze the results for the benign scenario, in which we consider different network delay distributions, aggregation functions, and number of NTP servers n, and will then analyze the different adversarial scenarios described above.

Benign scenario. Because realistic values of $\Delta^{\max}$ are within fractions of a second and have very little effect on this simulation, Figure 3 shows the results for a fixed value of $\Delta^{\max} = 0.05$ seconds. Naturally, combining time measurements from $n = 5$ NTP servers may lead to a larger uncertainty when a maximally transparent aggregation function is used than combining measurements from fewer NTP servers. This is clearly visible in the graphs for Δ^{TSP} in Figure 3, especially for network delay distributions that impose a significant likelihood of occasional outliers.

By design, the aggregation function $\mathcal{T}_1$ results in the smallest values of Δ^{TSP} across all simulations of maximally transparent aggregation functions since it sets t^{TSP} to the center of the interval $[t_{\min}, t_{\max}]$. However, the resulting error of t^{TSP} often far exceeds that of $\mathcal{T}_2$ and $\mathcal{T}_3$.

In terms of the absolute error of t^{TSP}, the aggregation function $\mathcal{T}_3$, which sets t^{TSP} to the median value of t^{NTP}, naturally results in the most accurate aggregation among the three defined maximally transparent functions. However, the associated values of Δ^{TSP} far exceed those of $\mathcal{T}_1$ and $\mathcal{T}_2$, especially in those simulations that emulate significant network delays, due to the potentially large difference between a single outlier and the median of the time measurements.

While the use of the basic outlier detection mechanism described above in the variants $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$ means that these aggregation functions are not maximally transparent, it leads to a significant reduction of both the error of t^{TSP} and of the uncertainty Δ^{TSP}. Of course, $\mathcal{T}_{1+Q}$ achieves the smallest values of Δ^{TSP}, but $\mathcal{T}_{2+Q}$ and $\mathcal{T}_{3+Q}$ also achieve a significant reduction in the uncertainty measure Δ^{TSP} when compared to $\mathcal{T}_2$ and $\mathcal{T}_3$, respectively. The absolute error of t^{TSP} is also reduced across all three variants, albeit not by much in the case of $\mathcal{T}_3$ / $\mathcal{T}_{3+Q}$ since outliers have very limited impact on the value of t^{TSP} in either case. Nevertheless, the significant decrease in uncertainty due to the removal of a single outlier may make $\mathcal{T}_{3+Q}$ an appealing candidate, especially in setups with more than three NTP servers.

Adversarial Scenario $\mathcal{A}_1$. Figure 4 depicts the same data for adversarial scenario $\mathcal{A}_1$ that Figure 3 depicts for the benign case. In this adversarial scenario, an adversary has successfully impersonated one of the NTP servers and is attempting to backdate the timestamp value t^{TSP} by sending an outdated time value t^{NTP}. By design, all *maximally transparent* functions remain faithful in any adversarial scenario that includes at least one honest NTP server. However, it is immediately apparent that the time value t^{TSP} as calculated by $\mathcal{T}_1$ is shifted by about five seconds regardless of the number of NTP servers n. This is because $\mathcal{T}_1$ considers only the bounds of the interval containing all time measurements when determining t^{TSP}, and the interval now includes a point in time that is approximately ten seconds before the true time. The aggregation function $\mathcal{T}_2$ is affected to a lesser extent since

it calculates the arithmetic mean of the time measurements, which means that the impact of the incorrect time value decreases as n grows: the absolute error of t^{TSP} is approximately $1/n \times 10$s, i.e., approximately 3.3s at $n = 3$ and 2s at $n = 5$. The aggregation function $\mathcal{T}_3$ is the least affected among the maximally transparent functions in adversarial scenario $\mathcal{A}_1$, which is expected because the malicious NTP server is providing a time that is, with high likelihood, far from the median of the time measurements and hence ignored for determining t^{TSP}. Because $\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$ are maximally transparent, we of course see a significant increase in the uncertainty Δ^{TSP} for all three functions. While $\mathcal{T}_1$ typically leads to an uncertainty Δ^{TSP} of about five seconds, which is the minimal uncertainty to cover the ten-second interval between the malicious time measurement and the true time, the uncertainty measure increases to about 10 seconds for $\mathcal{T}_3$, which corresponds to a 20-second interval centered, with high likelihood, around a time value that is close to t^{true}.

In this adversarial scenario, the variants with outlier removal via Dixon's Q test $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$ demonstrate additional value beyond the removal of rare benign outliers that we observed in the benign scenario. Based on Figure 4, across all tested values of n, we see lower absolute errors of t^{TSP} and lower uncertainty measures Δ^{TSP} for $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$ than for $\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$, respectively. Notably, for $n = 3$ and for those network delay distributions that occasionally result in significant network delays (B, C, E, and F), the upper quartile of the absolute error of t^{TSP} as well as the upper quartile of the uncertainty Δ^{TSP} are still large and often in the same range as the median values of the same metrics for the aggregation functions without outlier removal. For example, the upper quartile of Δ^{TSP} for $\mathcal{T}_{3+Q}$ is about 10 seconds for $n = 3$ with network delay distributions B, C, E, and F despite the use of outlier removal, but this is not the case for the delay distributions A and D nor for greater values of n. The reason is that if there are significant network delays, the benign time measurements may be spread out too far for Dixon's Q test to identify the malicious time measurement as an outlier. This explains the dependency of the behavior on the network delay distribution. The dependency on n is explained by the very large critical value $\text{Crit}(\alpha, n) = 0.970$ for $n = 3$ [55], which is much lower for larger values of n. The combination of the large critical value and the possibility that the benign samples are not close together means that neither Equation 1 nor Equation 2 might hold and no outlier is removed despite the adversarial input. At $n > 3$, we see that the outlier removal succeeds in discarding the malicious input with high likelihood, thereby obtaining very small errors in t^{TSP} and Δ^{TSP}.

Adversarial Scenario $\mathcal{A}_2$. In this scenario, an attacker has not only successfully impersonated one of the NTP servers and is again backdating its time responses by ten seconds, but is further delaying the requests to one of the other NTP servers by three seconds. Note that we are assuming a powerful attacker, who not only somehow holds valid NTS keys for one trusted NTP server but also has the ability to delay UDP requests to other NTP servers, typically by controlling a segment of the network route between the client and server, which in this case could be the untrusted host of the TEE. The attacker's benefit in delaying the request to one of the other NTP servers becomes apparent upon inspecting the data in Figure 5. By artificially causing larger differences between the benign samples, the attacker is able, to some extent, to conceal the fact that the response from the malicious NTP server is an outlier. In fact, for $n \in \{3, 4\}$, the aggregation functions that discard outliers ($\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$) behave almost identically to the base variants because the outlier conditions (Equation 1 and Equation 2) almost never hold. This results in large median uncertainties Δ^{TSP} of between 5 and 10 seconds across all six aggregation functions. Nevertheless, $\mathcal{T}_2$ and $\mathcal{T}_3$, as well as $\mathcal{T}_{2+Q}$ and $\mathcal{T}_{3+Q}$, still enjoy relatively small errors in t^{TSP}. At $n = 5$, we again see a larger benefit of outlier detection, with much lower errors of t^{TSP} and lower uncertainties Δ^{TSP} for $\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$ than for their base variants. This is because $\text{Crit}(\alpha, 5) = 0.710$ [55] is significantly lower than for $n < 5$, thereby allowing Dixon's Q test to correctly identify the backdated time as an outlier. When the malicious NTP response is discarded, only the delayed NTP request to an honest server increases the uncertainty Δ^{TSP} and the error of t^{TSP}.

Adversarial Scenario $\mathcal{A}_3$. Similar to the previous scenario $\mathcal{A}_2$, the adversary induces a delay into the NTP request to one of the benign NTP servers. However, instead of a short three second delay, the adversary in this scenario delays the request by 50 seconds. Because the adversary is also impersonating another NTP server that provides timestamps backdated by ten seconds, this means that the TEE will observe times as far apart as 60 seconds, with some variance due to simulated network delays and imperfect accuracies.

Figure 6 shows the absolute error of t^{TSP} and the uncertainty Δ^{TSP} for each of the aggregation functions in the same manner as before. Notably, at $n = 3$, the variants with outlier removal ($\mathcal{T}_{1+Q}$, $\mathcal{T}_{2+Q}$, and $\mathcal{T}_{3+Q}$) behave almost identically to the maximally transparent aggregation functions ($\mathcal{T}_1$, $\mathcal{T}_2$, and $\mathcal{T}_3$, respectively). This again is because the outlier detection does not reach the critical value required for detection and removal for either the backdated time or for the delayed NTP query. While $\mathcal{T}_1$ and $\mathcal{T}_2$ (and $\mathcal{T}_{1+Q}$ and $\mathcal{T}_{2+Q}$) are visibly impacted by the delayed response, no aggregation function leads to a backdated timestamp, and $\mathcal{T}_3$ (and $\mathcal{T}_{3+Q}$) even achieve a very small error of t^{TSP}. However, all aggregation functions necessarily lead to very large uncertainties ranging from the theoretical minimum of about 30 seconds in the case of $\mathcal{T}_1$ to about 50 seconds in the case of $\mathcal{T}_3$.

At $n = 4$, we begin seeing an effect that becomes fully visible at $n = 5$. As n increases, the critical value $\text{Crit}(\alpha, n)$

decreases [55], and there is an increasing likelihood that the delayed NTP query, which returns a time that is approximately 50 seconds later than the true time, is identified as an outlier and discarded in any of the non-maximally transparent aggregation functions. (Note that, without the backdated time obtained from the malicious NTP server, the delayed response from a benign NTP server would be identified as an outlier with high likelihood already at $n = 3$.) This demonstrates an interesting aspect of a publicly attested to policy for calculating timestamps: an on-path attacker that knows about the exact conditions for outlier removal may be able to get the TEE to discard benign NTP responses by delaying the associated NTP requests, thereby increasing the impact of the malicious NTP responses from an impersonated NTP server. Nevertheless, after the removal of the delayed benign NTP response as an outlier, the fact that the remaining samples are aggregated using a maximally transparent aggregation function means that the resulting timestamp remains faithful, even if $t^{\text{TSP}} \pm \Delta^{\text{TSP}}$ includes the backdated time. In the case of $\mathcal{T}_{1+Q}$, the value of t^{TSP} at $n = 5$ is very likely less than t^{true} because the adversary has successfully spoiled outlier removal by delaying a benign NTP response such that the benign response appear to be an outlier, however, the uncertainty measure Δ^{TSP} is large enough to still result in a truthful timestamp. Notably, the absolute error of t^{TSP} as determined by $\mathcal{T}_3$ and $\mathcal{T}_{3+Q}$ remains very low across all parameters of this scenario, and only the uncertainty measure Δ^{TSP} increases to a very large value, which however is remedied in the case of $\mathcal{T}_{3+Q}$ when n is increased to five NTP servers.

A.3 Summary

We have considered six different aggregation functions for computing a timestamp time and uncertainty from a set of NTP responses, half of which are maximally transparent while the other half removes at most one outlier before delegating to a maximally transparent aggregation function. Even in scenarios with a powerful adversary, who is not only impersonating one NTP server but also able to delay NTP requests to other NTP servers, no aggregation function resulted in backdated timestamps: even in the rare scenarios in which an adversary achieves a value of t^{TSP} that is less than the true time t^{true}, the TEE will correctly determine the uncertainty Δ^{TSP} to be sufficiently large such that the true time will almost certainly be within the interval $t^{\text{TSP}} \pm \Delta^{\text{TSP}}$. A robust, maximally transparent aggregation function combined with basic outlier detection – as implemented, for example, in $\mathcal{T}_{3+Q}$ – demonstrates promising results in the presence of network delays, imperfect accuracy of time sources, and adversarial behavior from a powerful attacker.

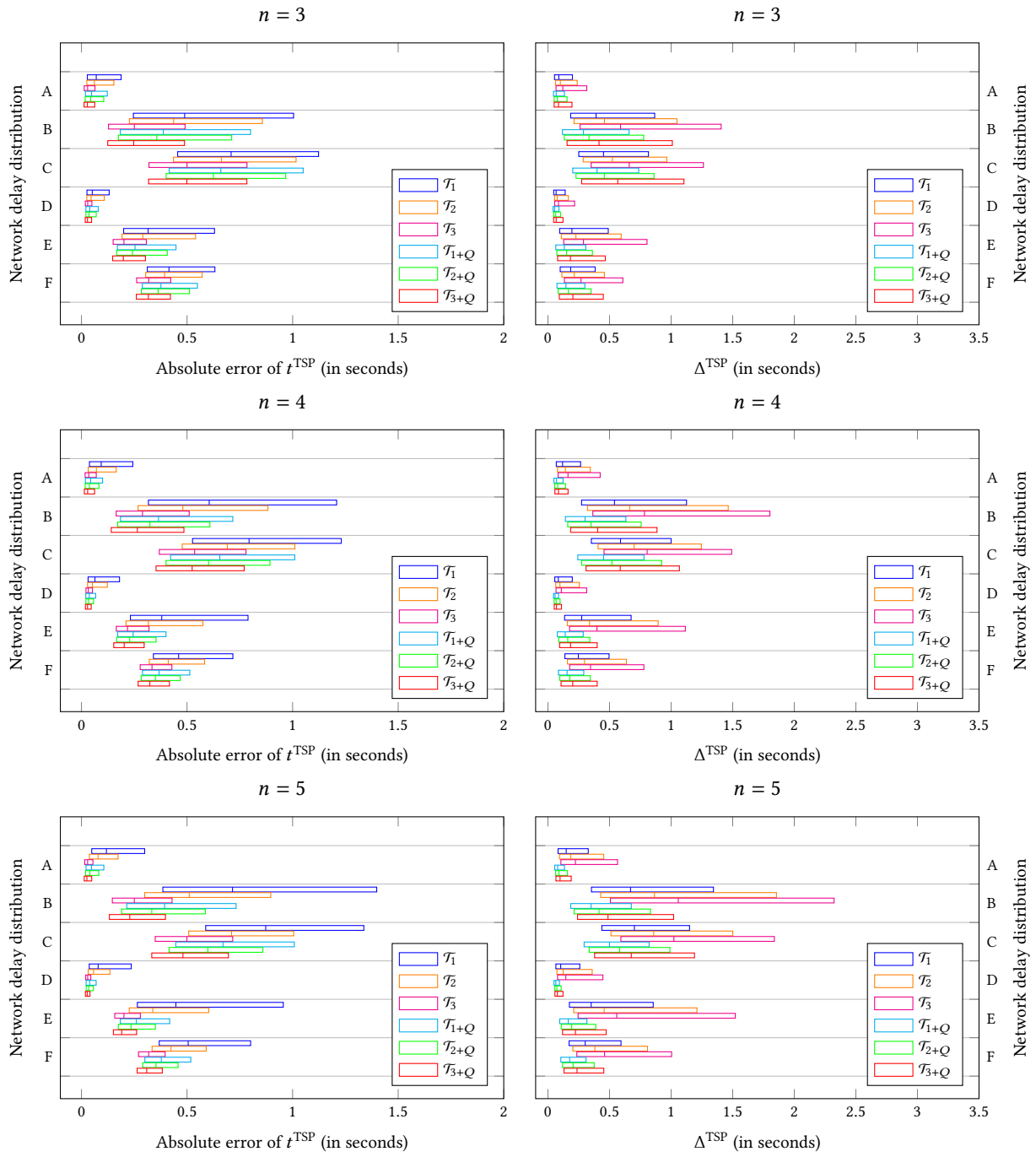

Figure 3. Simulation results in a benign setting for $\Delta^{\max} = 0.05$ seconds with a varying number n of NTP servers and varying network delay distributions (see Table 1). The left graph in each row depicts the absolute error of t^{TSP} and the right graph depicts the value of Δ^{TSP}, as standard box plot graphs (without whiskers).

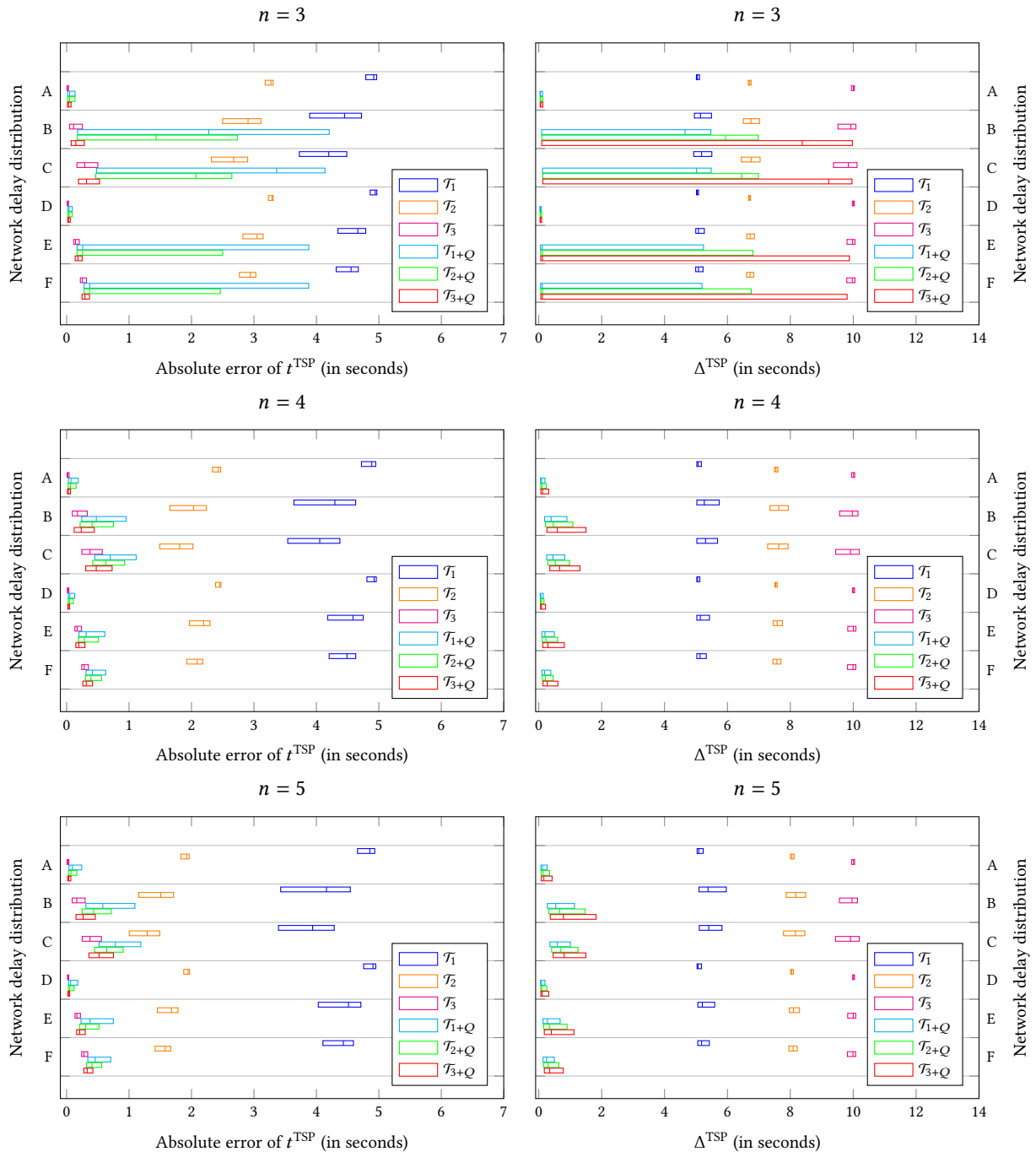

Figure 4. Simulation results in adversarial scenario $\mathcal{A}_1$ for $\Delta^{max} = 0.05$ seconds with a varying number n of NTP servers and varying network delay distributions (see Table 1). The left graph in each row depicts the absolute error of t^{TSP} and the right graph depicts the value of Δ^{TSP}, as standard box plot graphs (without whiskers).

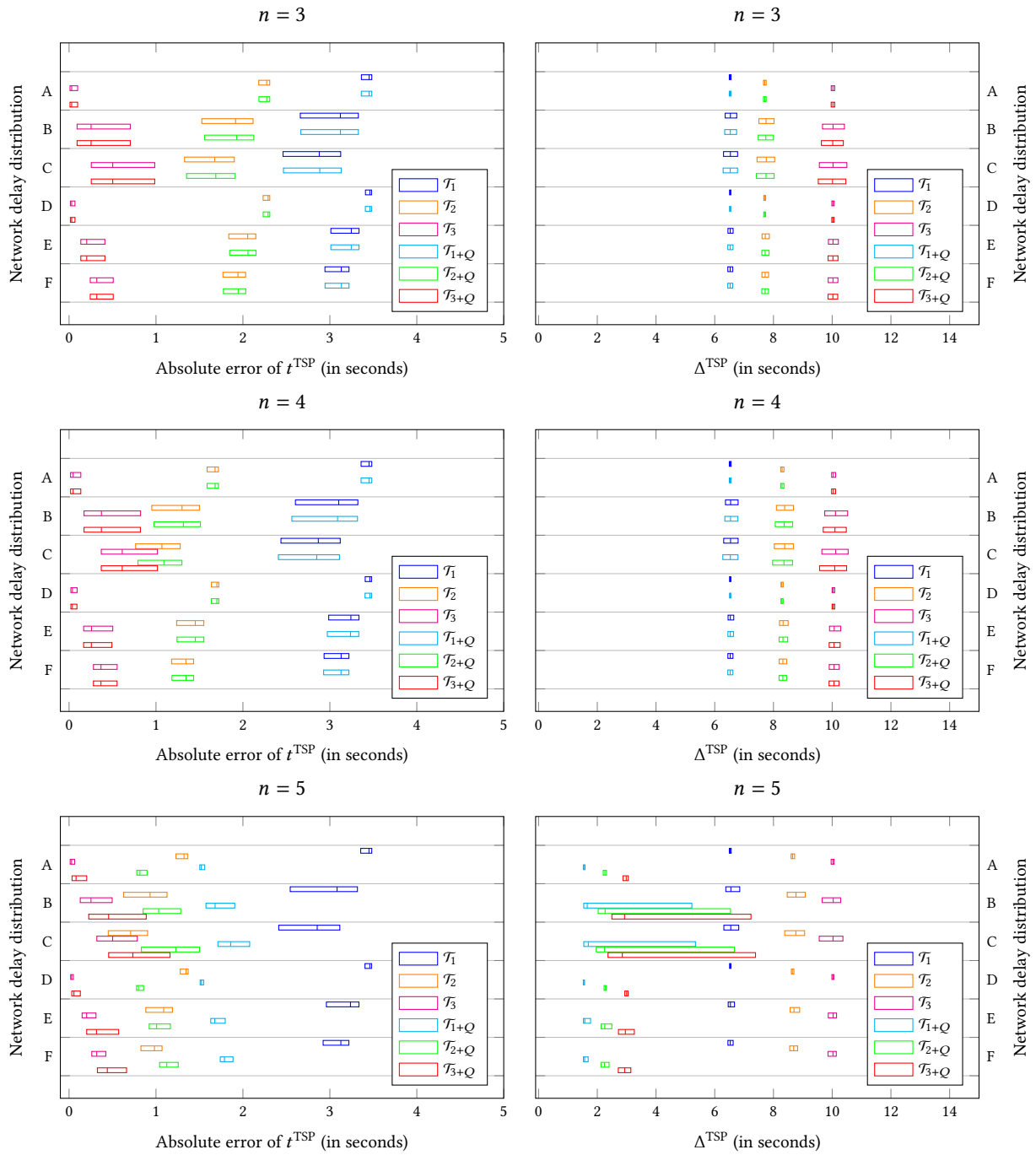

Figure 5. Simulation results in adversarial scenario $\mathcal{A}_2$ for $\Delta^{\max} = 0.05$ seconds with a varying number n of NTP servers and varying network delay distributions (see Table 1). The left graph in each row depicts the absolute error of t^{TSP} and the right graph depicts the value of Δ^{TSP}, as standard box plot graphs (without whiskers).

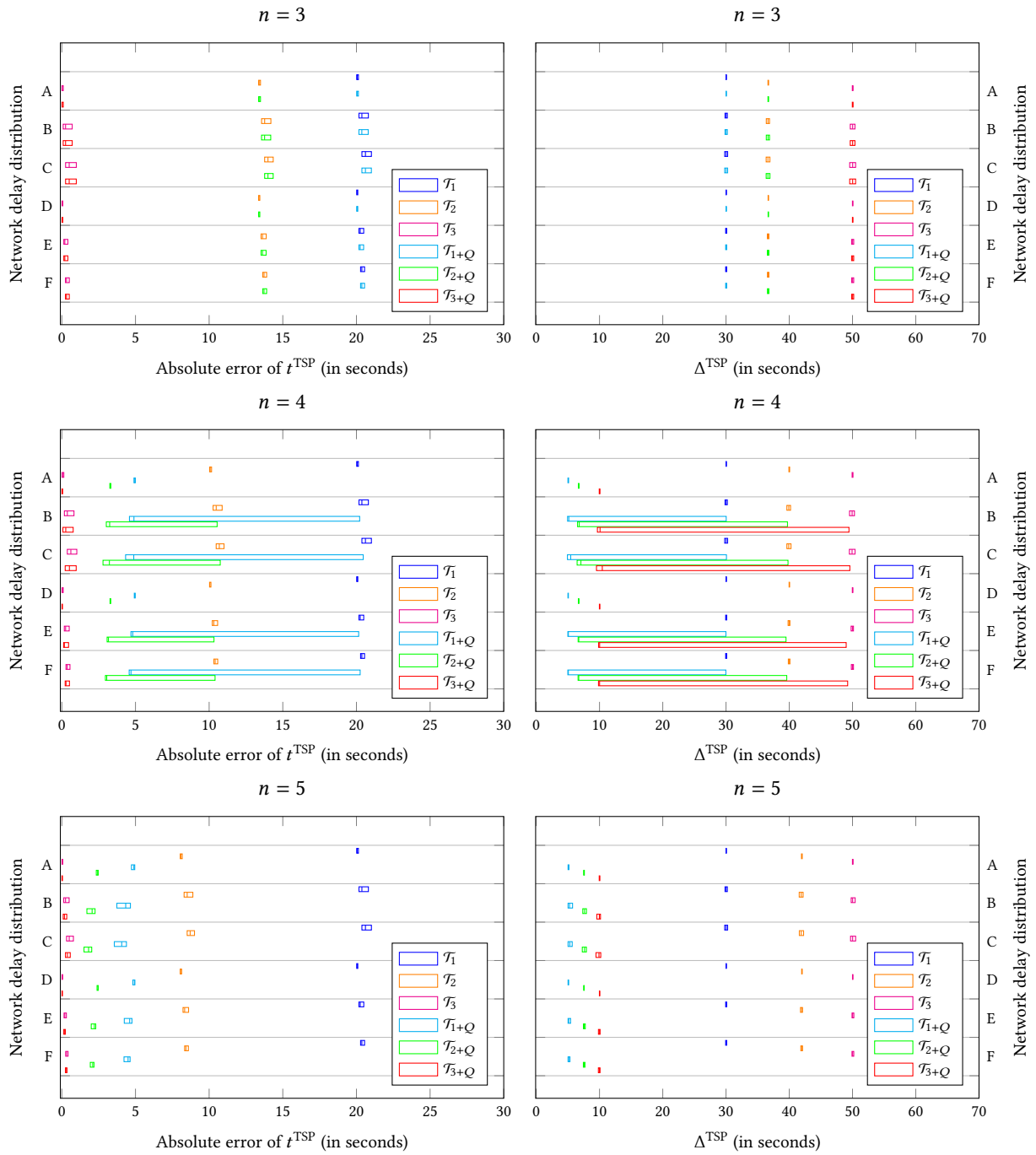

Figure 6. Simulation results in adversarial scenario $\mathcal{A}_3$ for $\Delta^{max} = 0.05$ seconds with a varying number n of NTP servers and varying network delay distributions (see Table 1). The left graph in each row depicts the absolute error of t^{TSP} and the right graph depicts the value of Δ^{TSP}, as standard box plot graphs (without whiskers).